Implementing Multi Linear Regression with Gradient Descent

DATA -527 Predictive Modeling, Spring 2025,

01March2025.

(Katie Elms, Srinila Pogalla, Siddartha Bandi)

1.Overview

The purpose of this project is to implement a **Multi Linear Regression (MLR)** model using the **Gradient Descent** algorithm. Multiple Linear Regression is an extension of simple linear regression the only difference is that multiple linear regression uses more than one slope (more than one independent variable). The main objective is to analyze the correlation between the independent variables (x_1, x_2) and the dependent variable (y) using a dataset "myDataMLR.csv" with 47 entries. The model implementation involves normalizing the data and training the model through iterative optimization, calculating the Mean Squared Error (MSE) for each iteration, and logging these values to track error reduction over time. Additionally, key model parameters, including the learning rate, number of iterations, slope (coefficient), intercept, R-squared value (R^2), and Mean Absolute Error (MAE), are computed and stored for further analysis. The model visually represents the data by plotting a vertical line over the actual and predicted scattered data points and a separate graph illustrating the MSE trend per iteration. This project provides a deeper understanding of how multi linear regression works mathematically and demonstrates the effectiveness of Gradient Descent in optimizing model parameters for predictive analysis.

2.Methodology

The primary goal of this project is to construct a Multi Linear Regression model using mathematical equations to analyze the relationship between an independent variables and dependent variable. To optimize model performance, Gradient Descent, a widely used optimization algorithm, is used. This algorithm iteratively minimizes the error between predicted and actual values, ensuring the model learns effectively. The steps in methodology includes

i. **Collection of Data:** The Dataset myDataMLR.csv contains three attributes x_1, x_2 and y. Each row in the dataset represents a unique observation, and the dataset comprises a total of 47 observations.

ii. **Data Preprocessing:** The process where the raw data is processed and transformed in to a suitable format for further analysis. The data is normalized using Min-Max scaling and converted in to Numpy arrays. Add bias term to the feature matrix to enhance linear regression modeling

iii. **Building Multi Linear Regression model with Gradient Descent:** Steps to build and train a Multi linear regression model with gradient descent algorithm

- Initialize the model parameters and variables like slope, intercept, number of iterations and learning rate.
- For each iteration, calculate the predicted output of each observation.
- Calculate the mean square error to evaluate the error.
- Calculate the derivatives with respective to each model parameter.
- Update the parameters using the learning rate and the gradient.
- Calculate model parameters including mean square error, slope, intercept and R squared value (R^2), mean absolute error.
- Save and Print the model parameters.

iv. **Visualization:** After calculating the metrics, a vertical line is plotted between the scatter plot of actual and precited values. Moreover, a MSE per iteration graph is used to observe the error reduction per each iteration.

3.Implementation

i. **Import necessary libraries:** NumPy library is used for numerical computations, Pandas helps in reading and handling datasets, Matplotlib is used for visualizing the data, Argparse is used to write command-line interfaces.

ii. **Data Loading and Preprocessing:** Reads the dataset from myDataMLR.csv using pd.read_csv and extracts the independent variables (x_1 and x_2) and dependent variable (y). These features are normalized using Min-Max Scaling to convert values in the range [0, 1].

$$\mathbf{X}_{\text{normalization}} = \frac{x - x\,min}{xmax - \text{xmin}}$$

iii. **Model Initialization:** Initialize the Learning rate (learning_ rate), number of iterations (iterations) through the command line argument and set model parameters (theta) to zero.

iv. **Gradient Descent Algorithm:** For each iteration in iterations

a. Calculate the Predicted 'y' values (prediction) using:

$$\mathbf{Y}_{\text{pred}} = \mathbf{X}.\ \boldsymbol{\theta}$$

Where,

X=Feature Matrix

Θ= parameter vector

b. Calculate the Mean Squared error (MSE) and store the values in a log file (MSE_log)

$$\textbf{MSE=1/2m} \sum_{i=1}^{m} (yi - h(xi))2$$

h (x) = intercept + slope. x1 + slope. x2

Whereas,

x1, x2 are independent variables.

m = size of data,

h(x) = predicted value

c. Calculating the derivatives for slope and intercept:

$$\text{slopePartialDerivative}_1 = \frac{-1}{m}\sum_{i=1}^{m}\left(y^i - h(x^i)\right) \times x_1^i$$

$$\text{slopePartialDerivative}_2 = \frac{-1}{m}\sum_{i=1}^{m}\left(y^i - h(x^i)\right) \times x_2^i$$

$$\text{slopePartialDerivative}_n = \frac{-1}{m}\sum_{i=1}^{m}\left(y^i - h(x^i)\right) \times x_n^i$$

$$\text{interceptPartialDerivative} = \frac{-1}{m}\sum_{i=1}^{m}\left(y^i - h(x^i)\right)$$

d. Update the slope and intercept:

$$\text{slope}_1 = \text{previousSlope}_1 - (L \times \text{slopePartialDerivative}_1)$$

$$\text{intercept} = \text{previousIntercept} - (L \times \text{interceptPartialDerivative})$$

e. Compute final model parameters:

a. Calculate the final predicted 'y' values:

y_pred=x. θ

b. The accuracy of the model is calculated by R^2:

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

v. **Save Model Data:**

   a. Save Mean Square Error log file (MSE_ log) to a text file “MLR Training_ Iterations_ Learning Rate_ MSE.txt”

   b. Save model parameters (learning_rate, iterations, slope, intercept, R^2 to “MLRModelParameters.txt”.

vi. **Visualize Results:**

   a. Plot the vertical line over the scattered datapoints of actual and predicted datapoints and save as “MLR_Actual_vs_Predicted_plot.png”.

   b. Plot Mean Square Error and save as “MLR_MSE_Iteration.png”

Pseudocode

```
BEGIN

# Parse command-line arguments (learning_rate, iterations) or prompt user input

   SET learning_rate = ARGUMENT OR USER INPUT

   SET iterations = ARGUMENT OR USER INPUT

 # Load and normalize dataset

   READ dataset "myDataMLR.csv"

   NORMALIZE [x1, x2, y] using Min-Max Scaling

 # Prepare feature matrix X and target variable y

   X = [ones_column, x1, x2]

   y = column [y]

   n =Number of samples



   # Function: Compute Mean Squared Error (MSE)

   FUNCTION compute_mse(X, y, theta)

      RETURN MEAN ((y - (X * theta)) ^ 2)

   ENDFUNCTION
```

```
# Function: Perform Gradient Descent
    FUNCTION gradient_descent(X, y, learning_rate, iterations)
        INITIALIZE theta AS ZEROS
        INITIALIZE mse_log AS EMPTY LIST
 FOR i FROM 1 TO num_iterations DO
            PREDICTIONS = X * theta
            GRADIENTS = (2/m) * TRANSPOSE(X) * (PREDICTIONS - y)
            UPDATE theta = theta - (learning_rate * gradients)
 # Compute and log MSE
            APPEND compute_mse(X, y, theta) TO mse_log
        ENDFOR
   RETURN theta, mse_log
    ENDFUNCTION
# Train model
    theta, mse_log = gradient_descent(X, y, learning_rate, iterations)
# Compute R-squared
    PREDICTIONS = X * theta
    R_SQUARED = 1 - (SUM((y - PREDICTIONS) ^ 2) / SUM((y - MEAN(y)) ^ 2))
# Save and display model parameters
    PRINT "Learning Rate:", learning_rate
    PRINT "Iterations:", iterations
    PRINT "Final MSE:", mse_log[-1]
    PRINT "Intercept:", theta [0], "Slope x1:", theta [1], "Slope x2:", theta [2]
    PRINT "R-squared:", R_SQUARED
```

```
# Plot MSE over iterations
    PLOT iterations vs. mse_log
    SAVE PLOT "MLR_MSE_Iteration.png"
END
```

4.Results:

Learning Rate: 0.1

Iterations: 500

Final MSE: 4163158586.021152

Intercept: 182852.25040273753

Slope x1: 465736.4633247465

Slope x2: 18020.407885292734

R-squared: 0.7279393392979494

The multiple linear regression model, with a learning rate of 0.1 and 500 iterations, resulted in a final mean squared error (MSE) of 4,163,158,586.02, indicating a significant gap between the predicted and actual values. The intercept (182,852.25) represents the baseline value of the dependent variable when both independent variables are zero. The coefficients for x_1 (465,736.46) and x_2 (18,020.41) make contributions to the model, with x_1 exerting a much stronger effect on the predictions. The R-squared value of 0.7279 suggests that approximately 72.79% of the variance in the dependent variable is accounted for by the model, reflecting a moderate fit. However, the relatively high MSE indicates potential issues, such as the presence of outliers, inadequate feature scaling.

Learning Rate	Iterations	Final MSE	Intercept	Slope x1	Slope x2	R-squared
0.1	500	4163158586.02	182852.25	465736.46	18020.40	0.7279
0.1	1000	4089307761.43	196128.72	497677.75	-24754.45	0.7327
0.01	500	7120830022.82	205601.75	199650.96	137127.23	0.5346
0.01	1000	5748739543.18	184853.76	278567.51	126709.89	0.6243
0.001	500	16934162201.90	180210.39	76424.59	104762.02	-0.1066
0.001	1000	9796269601.60	220503.19	105019.55	130537.20	0.3598

Table 1: Mean Square Error for different learning rates and iterations

The table illustrates how different learning rates and iteration counts influence the effectiveness of a Multi-Linear Regression model trained using Gradient Descent. A higher learning rate (0.1) accelerates error reduction but can cause instability, leading to fluctuating parameter values and occasional negative coefficients. In contrast, a moderate learning rate (0.01) ensures a more stable learning process, progressively lowering the Mean Squared Error (MSE) while maintaining a higher R-squared value, which reflects a better model fit. A very low learning rate (0.001) results in higher MSE, and poor R-squared values, demonstrating the model's difficulty in capturing patterns effectively. Increasing iterations from 500 to 1000 generally improves performance, but the benefits diminish over time as the model nears convergence. The most balanced approach is observed at a learning rate of 0.01 with at least 1000 iterations. However, an excessively high learning rate may cause the model to overshoot optimal parameter values, leading to unpredictable behaviour. Ultimately, selecting an appropriate learning rate is essential for ensuring the model learns efficiently and accurately.

.

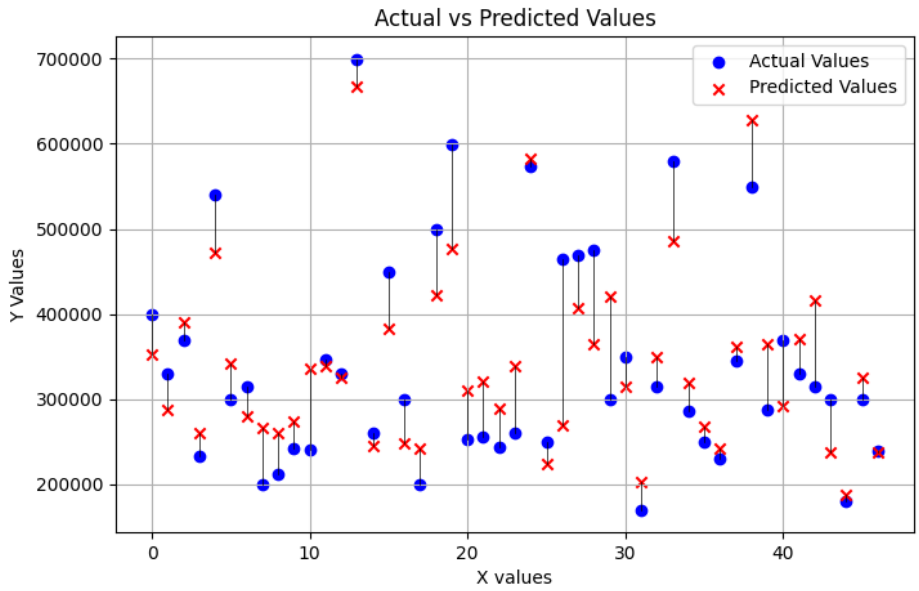

Fig 1: The Scattered plot of Actual and Predicted values

This Scattered plot visualizes the comparison between the actual and predicted values of the dataset. Blue denotes the actual values and red crosses denotes predicted values and the vertical line measures the error between the predicted and actual values

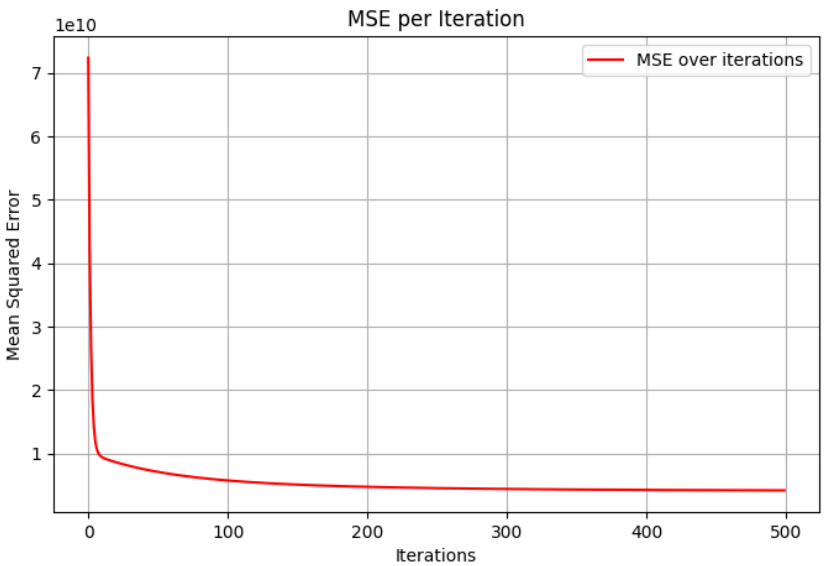

Fig 2: The graph of MSE per iteration

The above graph shows how the Mean Square Error decreases after each iteration. The curve starts at a high error and decreases sharply at first, then flattens out as the model converges.

5.Discussion:

During the implementation of the multi-linear regression model, several challenges were encountered and addressed. One major issue was handling command-line arguments in Jupyter Notebook, where argparse caused errors. This was resolved by using parse_known_args() to ignore unrecognized arguments. Another challenge was feature scaling, as large variations in feature values affected the stability of gradient descent. To address this, Min-Max scaling was applied to normalize the data.

Different learning rates (0.001, 0.01, 0.1) were tested to ensure stable convergence without overshooting. Logging and visualizing Mean Squared Error (MSE) over iterations was also necessary to track model performance. MSE values were stored in a file for reference, and Matplotlib was used to plot error trends. Additionally, evaluating the model required more than just MSE, so the R-squared metric was computed to measure the goodness of fit.

Some parts of the implementation were relatively straightforward, Python library like Pandas, allows easy feature scaling and conversion to NumPy arrays. Matrix computations were efficiently handled using NumPy's dot product, making predictions and updates faster. Visualization with Matplotlib was intuitive, enabling quick analysis of trends and comparisons. Lastly, logging model parameters in a text file provided an organized way to track results. These streamlined steps helped in achieving a structured and effective implementation of the model.

6.Conclusion:

This project successfully implemented a Multi-Linear Regression model optimized using Gradient Descent provides key insights to enhance model performance through feature scaling, learning rate optimization, and iterative parameter updates. The results demonstrate the importance of hyperparameter selection and normalization in achieving efficient and accurate predictions. Although challenges arose in hyperparameter tuning and error analysis, the model effectively identified patterns and reduced errors over successive iterations. The outcomes highlight the significance of proper data preprocessing and optimization strategies in developing a precise and efficient regression model.

7.Reference:

1. Dataset: myDataMLR.CSV
2. "Multiple Linear Regression" notes by Dr. Fatima Kuehn
3.https://www.scribbr.com/statistics/multiple-linear-regression/#:~:text=What%20is%20multiple%20linear%20regression,variables%20using%20a%20straight%20line.